addyosmani.com

Agentic Code Review

Addy Osmani

26–34 minutes

Coding agents are extraordinarily good now, and getting better fast. The interesting consequence is that the hard part of engineering moved from writing code to deciding whether to trust it, which makes review the most leveraged skill in software right now. How you approach it depends enormously on who you are: a solo developer with no users and a team maintaining a ten-year-old application are not solving the same problem.

I am more optimistic about agentic engineering than I have ever been. The agents are genuinely good, they get better every month, and on an ordinary day I now ship things I would not have attempted a year ago. This write-up is a map of where the interesting work went, because it did move, and most teams have not fully caught up to where.

Code review used to work because of a happy accident of relative speed. A senior engineer could read code faster than a junior could write it, so review kept pace without anyone designing it to, and the team absorbed how the system fit together as a side effect of reading each other's diffs. A lot of that was not deliberate. It fell out of a single fact: writing code was the slow, expensive part, and reading it was cheap and fast.

That fact no longer holds. An agent will produce a thousand lines of often solid, well-formatted code in less time than it takes me to read this paragraph, while a human's reading speed has not changed since roughly the day we started staring at screens for a living. So the constraint moved downstream, to the one step that did not get faster: a person being confident the change is right. I do not think that is a loss. It is the most leveraged place in software to be good right now, and it is where I have put most of my attention this year.

There is a happy twist here that shapes the rest of this piece. The same tools generating all that extra code are also the best thing I have for keeping up with it. On my own projects, including the popular open-source ones, I now point Claude Code or Codex at a batch of incoming PRs and have them triage the queue for me, and that has genuinely changed how I spend my time. So this is not an anti-AI argument, and I will come back to exactly how I use it.

It is also not a data dump, and not another round of whether letting a model write your code is wonderful or the end of the craft, because that framing is useless. The only answer that survives contact with a real codebase is that it depends entirely on who you are. A developer vibe-coding a side project a dozen people will ever run, and a team keeping a ten-year-old enterprise system alive for another quarter, share almost no constraints worth naming, and most of the advice in circulation is really one of those two people telling the other how to live.

What the 2026 data actually shows

The productivity gains from AI are real, but raw output

overstates them: about four times the code for a tenth more delivered value. The gap between those numbers is review work, which is exactly why review is where the leverage now sits.

For a couple of years this was anecdote and argument. It is now measured at scale, by organizations with no shared agenda and in several cases competing commercial interests, and the measurements keep pointing the same way: AI pushes output sharply up, and pushes both quality and reviewability down.

Faros AI instrumented 22,000 developers across 4,000 teams and tracked what happened as teams moved from low to high AI adoption. This is March 2026 data, about as current as anything here. The upside is real and worth stating plainly: developers merge considerably more PRs and complete more work, and throughput per engineer climbs. Then the rest of the report:

- code churn up **861%**
- the incidents-to-PR ratio up **242.7%**
- the per-developer defect rate up from **9% to 54%**
- median review *duration* up **441.5%**, with time-to-first-review and average review time both roughly doubling
- PRs merged with **zero review up 31.3%**

  The last figure is the one I find hardest to dismiss, because nobody chose it. There was no decision to stop reviewing. Reviewers simply could not keep pace with the volume, so code began merging unread, and that became normal. The detail I keep returning to is that teams with mature, disciplined engineering practices were hit just as hard as

everyone else. Good process did not protect them, because the volume arrived faster than any process was designed to absorb.

One caveat to hold throughout: CodeRabbit and Faros both sell into this market, so their framing is not disinterested. That does not make the numbers wrong, the effect sizes are large and consistent across unrelated sources, but vendor research deserves to be read with that in mind.

CodeRabbit studied 470 open source PRs in December 2025, 320 AI-coauthored and 150 human-only, and found the AI changes carried roughly **1.7x more issues**: logic and correctness problems up about 75%, security issues 1.5 to 2x more common, readability problems more than tripling. Their AI director David Loker described these as "predictable, measurable weaknesses that organizations must actively mitigate". Predictable is the operative word. These are known, locatable weaknesses, which is good news: it means a review process, human or automated, can be aimed straight at them.

GitClear has the single number I would lead with. In their productivity data through 2025, daily AI users produce around **4x the raw output** of non-users, but measured against their own output a year earlier, the real productivity gain is only about **12%**. You are generating roughly four times the code for something like a tenth more delivered value, and a human still has to review all four times of it. To GitClear's credit, Bill Harding is explicit that some of even that 12% is selection bias, because stronger developers concentrated in the AI cohort. The gap between 4x the code and a tenth more value is the review problem stated in one line.

GitHub reports that Copilot review has now run over 60 million reviews, a 10x increase in under a year, and more than one in five reviews on the platform involves an agent. This is no longer a niche practice. It is how code gets made.

Four datasets, four methods, one conclusion. We poured machine-speed output into a system built for human-speed work. The bottleneck did not disappear; it moved to verification, and review is where that bill comes due.

Everyone is solving a different problem

How much review a change needs depends almost entirely on its blast radius, and most advice you read was written by someone operating at a very different one.

Almost all the alarming data above comes from enterprise telemetry and from open source maintainers being overwhelmed. It is entirely real if that is your situation. If you are one person shipping something a handful of people will ever run, much of it simply does not apply to you, and you should not be made to feel otherwise.

Three variables determine where you sit:

- **blast radius**: what happens when it breaks. Nothing, or angry users and money and PII on the line.
- **how long the code lives**: a throwaway prototype you might rewrite next week, or a codebase you will maintain for years.
- **how many people need to understand it**: just you holding the whole thing in your head, or a team that has to share ownership over time.

Run the same diff through those three and “good review” means genuinely different things.

If you are working solo on a greenfield project with no users, review's second job, distributing knowledge across a team, does not exist for you. You are the team. The reasonable move is to lean hard on tests and automation, review the parts that genuinely matter, and accept a lighter touch on the rest. Duplication and churn cost far less when the code may not exist in a month and nobody is paged at 3am when it breaks. The catch, and people learn this one painfully, is that it only works if the tests are real. Skipping review without a safety net does not remove the work, it defers it at a higher price, and standards slip when no one is there to push back. No users is permission to defer review. It is not permission to skip verification.

Then the project gets users. This is the dangerous middle, and the crossing is rarely noticed at the time. Review's bug-catching role suddenly matters, because bugs now hurt people, and its knowledge-sharing role switches on, because it is no longer only you. Teams keep their solo-era habits a few months too long, and then there is a postmortem and the Faros numbers stop being a chart and become their own dashboard.

At the far end is the large organization with an old codebase and many users. Here every alarming figure lands at full strength. A duplicated helper is not a style nit, it is a future bug surface and a maintenance cost that compounds for years. A change nobody understood is comprehension debt that becomes someone's on-call incident. Review is doing several jobs at once, and the volume of agent output quietly breaks all of them. The Faros finding about mature teams is aimed squarely here.

So the point is not "enterprises should be cautious and solo

developers can relax". It is that the purpose of review changes with your position, so the rules have to change with it. Bolt an enterprise's locked-down, multi-agent, evidence-required pipeline onto a two-person prototype and you have added friction for no benefit. Run "tests pass, ship it" on a payments system and you have built an incident generator with a green checkmark on top. Most bad advice in this space is one position on that spectrum prescribing to another.

What review is actually for now

Review was built to check an author's reasoning. An agent does reason, but that reasoning is usually thrown away rather than attached to the code, so the reviewer has to reconstruct a rationale that never made it into the diff. The good news: that is a tooling problem, and capturing the reasoning makes review dramatically easier.

This is the part that genuinely changed, and I think it is underappreciated.

When a human writes code, intent comes along for free. The reasoning, the alternatives weighed and discarded, lived in the author's head, and review was you checking that reasoning. Modern agents do reason, often visibly, producing thinking traces and weighing options and explaining themselves as they go. The catch is that this reasoning is usually discarded the moment the diff is produced. It is rarely captured, rarely attached to the PR, and in any case it is the agent's reasoning about how to implement the task, not a human's judgment about whether it was the right task to begin with. So review shifts from

checking reasoning that sits in front of you to reconstructing intent that never got written down, which is harder and slower, and we keep acting surprised that it takes 441% longer.

A 2026 paper, AI Slop and the Software Commons, analyzed 1,154 posts across 15 Reddit and Hacker News threads where developers discussed "AI slop". One line from a developer has stayed with me: reviewing an agent's PR made them "the first human being to ever lay eyes on this code".

That is worth sitting with, and it points straight at the fix. In normal review the author already understood the change and you were checking their work. With an agent PR, nobody has reconstructed the why yet, and the reviewer is the first to try. As the paper puts it, review "wasn't built to recover missing intent". The encouraging part is that missing intent is recoverable: the reasoning existed, we just discarded it. Have the agent state what it was trying to do and what it ruled out, capture that as a decision log on the PR, and a large part of the reconstruction cost disappears. This is a tooling problem, and tooling problems get solved.

None of which makes "have the AI review the AI" a complete answer on its own. A second model with different priors genuinely catches real bugs, and it catches a lot of them, which is why you should run one. What it does not supply is the human judgment about whether this is the right change to build in the first place. That judgment stays with a person, and it happens to be the most interesting part of the job, the part worth keeping.

The tools are good, but not always for the

reason they advertise

The current AI reviewers are genuinely good, and they occasionally don't flag the same lines as each other, so the right move is not picking the best one but running two that are built differently.

The dedicated AI review tools are good now, and I think you should be running at least one on everything, side projects included. CodeRabbit is the most widely deployed and topped the independent Martian benchmark (January to February 2026) on F1, around 49% precision with the best recall in the field. Greptile trades precision for recall: around an 82% bug-catch rate against CodeRabbit's 44% in one benchmark, at the cost of more false positives. Anthropic's Code Review reports under 1% of its findings marked incorrect by their engineers, and the figure I would actually show a manager: it raised their internal rate of PRs receiving a substantive review from 16% to 54%. The long tail of changes that used to get a glance and an approval now gets read by something.

The most useful result I have seen this year is not from a vendor. An engineer ran four reviewers in parallel, CodeRabbit, Sentry Seer, Greptile and Cursor BugBot, across 146 real PRs and 679 findings over three and a half weeks:

> Of 617 distinct flagged locations, **93.4% were caught by exactly one of the four tools**. 6% by two. Almost none by three. **None at all by all four.**

The four tools never once flagged the same line. Each was strong at a different class of problem: Greptile with near-zero false positives on correctness and architecture, CodeRabbit with the widest net and one-click fixes, Seer best

on production-failure severity. That is the adversarial review argument demonstrated on a real codebase rather than in a paper. Heterogeneity is the whole point. Four copies of one model is a single reviewer with a larger invoice, whereas four genuinely different reviewers surface a set of bugs no single member could find alone, the human included.

In practice: do not agonize over the single best tool, there isn't one. At the high-stakes end, run two with deliberately different characters (the experiment above paired Greptile for everyday correctness with Seer for production-failure severity, with almost no overlap). If you are solo, one good reviewer plus real tests is plenty. And whatever the marketing says, measure it on your own code, because every one of these results was specific to a particular codebase, and yours will be too.

Should we just let AI review more of it?

The machine is already reviewing more of your code than you are. The only real decision left is whether you do that deliberately, and the amount of human you keep should scale with your blast radius.

I keep hearing a question that would have been heresy a year ago, now from experienced engineers: should the machine be doing more of the reviewing, perhaps most of it? I no longer think that is a foolish question.

The uncomfortable part is that AI review works. Under 1% of Anthropic's findings are marked wrong, the tools catch bugs humans read straight past, and they do not get tired on the thirtieth PR of the day, which is exactly when a human is least reliable. Meanwhile humans are visibly not keeping up:

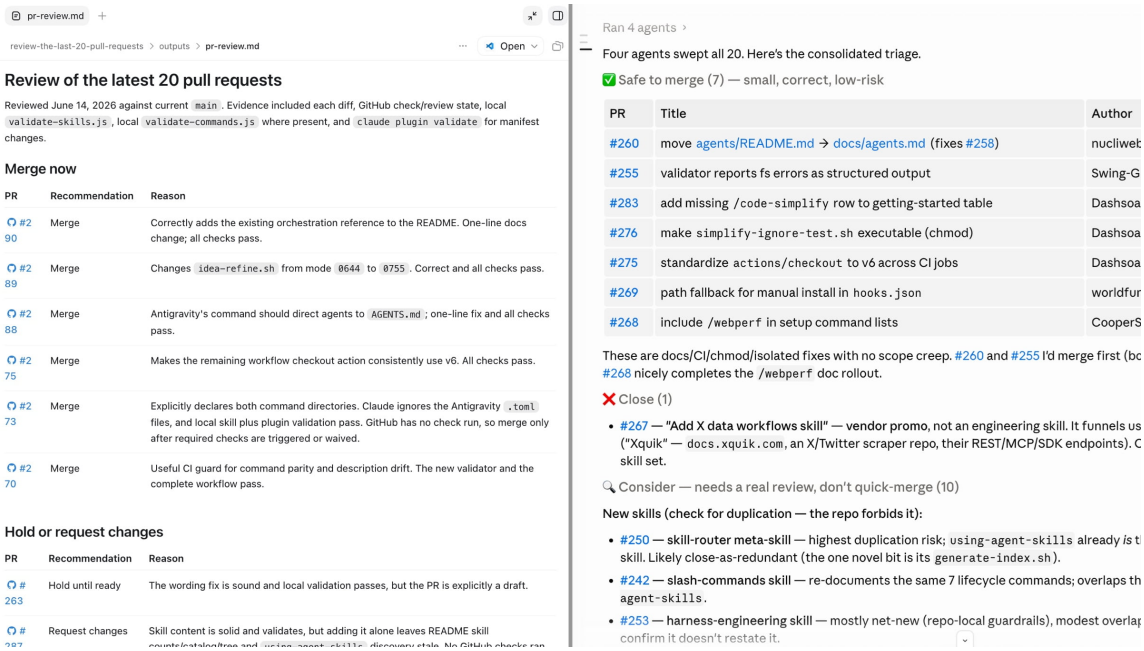
zero-review merges are up 31% and review times are up triple digits. In a real sense the machine is already reviewing more of the code than we are. The honest framing is not “should we let AI review more” but “AI is already doing it, are we going to be deliberate about that or let it happen by default while pretending humans still read everything”.

Loop engineering sharpens this. The premise of a loop is that you stop being the person who prompts the agent and instead build a system that prompts it, and a central part of that system is a judge: an agent that decides whether the work is done before moving on. The reviewer is the next role being designed out of the inner loop, on purpose. We spent a year automating the writing, and the loops are now automating the checking, and the human keeps getting pushed up and out. “Where does the human stay” is not a seminar question, it is something you decide every time you wire up a loop, whether or not you realize you are deciding it.

Where I currently land, and I hold this loosely: the answer is not “a human reads every line”. That is over. The volume ended it, and anyone insisting otherwise is describing a world that no longer exists. But it is also not “let the loop review itself and walk away”. When an agent writes the code, another reviews it and a third judges it, you have a closed loop of models with broadly correlated blind spots, especially when they come from the same family, confidently agreeing in the same places. A confident “looks good” with no human anywhere in it is borrowed confidence: the system’s certainty becomes yours, and nobody actually understood anything. The loop can be both very sure and very wrong, with no human left to tell the difference.

So the human does not leave; the human moves up a level. You stop reviewing every diff and start owning the parts that do not transfer to a model. Accountability, because you cannot page a model at 3am. The judgment of whether this is even the right change to build, as distinct from whether the code is correct. The high-blast-radius gates where being wrong is expensive. And the awkward one: the behavior nobody specified, because a model reviews the code that exists and rarely flags the requirement that nobody thought to write down, which remains a human-shaped gap I do not expect to close soon. Human in the loop becomes human on the loop: sampling, spot-checking and auditing the system rather than reading every PR, and spending your limited attention where being wrong would actually hurt.

This is already how I work on my own projects, including the open-source ones that now see more PRs in a day than I could carefully read in an evening. I point Claude Code or Codex at a batch of incoming PRs and ask for a first pass: a high-level read of what looks safe to merge, what needs more work, and what is genuinely high-risk. I do not auto-merge on the result, and I do not lazy-merge whatever it approves. What it gives me is a way to allocate attention. I can spend a few minutes confirming the changes it considers low-risk, and put real, careful time into the ones it flags as dangerous. The detail that matters is that this is not my old review hour made slightly faster. It is a different shape of hour, and at the volume I now deal with, it is the main reason the queue stays survivable at all.



Codex and Claude Code giving me a first-pass, risk-sorted read of a batch of PRs. The triage is the help. The merge decision stays mine.

A more extreme version of the same move is Kun Chen, an ex-Meta L8 engineer now shipping around 40 PRs a day as a solo builder, who has largely stopped reviewing code. It would be easy to dismiss this, except he is an L8, unusually good at the thing he stopped doing, which is what makes it interesting. He runs 20 to 30 agents in parallel and has moved his effort into the plan: he writes detailed plans up front, the agents run for hours against them, and he says plan quality determines how long they can run unattended. That is the move I described above in its purest form. It is worth being precise about what actually happened, because it is not that he stopped verifying. The intent did not vanish, he wrote it down himself in the plan, so the "first human to ever lay eyes on this" problem is half-solved: a human did understand the why, just up front rather than after. And he did not work without a net, he built an automated review gate (he calls it No Mistakes) that checks the code before it merges, and he stays on escalation when an agent gets

stuck. The human does the expensive thinking before the code exists and the machine does the line-by-line afterward, which may well be the shape of where this goes.

But he is a solo builder with no large team and no decade-old system full of landmines beneath him. The exact conditions that make 40 PRs a day without review rational for him are conditions most readers do not have. Copy his workflow onto a team shipping to many users and you reproduce the Faros numbers on your own dashboard. He is not wrong; he is a long way down one specific end of the spectrum.

Which is the spectrum point again. Solo with no users: letting AI review almost all of it is a defensible 2026 position, and you should not feel guilty about it. Maintaining something large for many people: let the machine handle the first pass, the second pass and the boring 90%, but keep a real human on the load-bearing paths and do not let the loop close completely on anything that can hurt someone. How much human you keep is a dial, and you set it by blast radius, not by guilt.

What to actually do

Stop reviewing everything to the same depth. Spend scarce human attention only where being wrong is costly, and let cheap deterministic gates and AI reviewers handle the rest.

The organizing idea is to match review effort to the cost of being wrong, push the cheap deterministic work as early as possible, and reserve human attention for what only humans can do.

Tier by risk, not by author. A config change earns a linter and a glance. A payments path earns the full stack: types, tests, two different AI reviewers, a human who owns that system, and a security pass. Do not spend a heavy review on boilerplate, and do not wave through an auth change because the tests are green. The layered approach is the same everywhere; what changes is how many layers a given diff has to clear.

Fast-fail the expensive tail. The most useful recent finding for teams drowning in agent PRs is Early-Stage Prediction of Review Effort (January 2026), which studied 33,707 agent-authored PRs. Agents are good at small, well-defined changes, around 28% merge almost instantly, but they tend to "ghost" the moment they get subjective feedback, abandoning the back-and-forth that review actually is. (A companion 2026 paper found reviewer abandonment accounted for 38% of rejected agent PRs.) The researchers built a "circuit breaker" that predicts high-maintenance PRs from cheap signals like file types and patch size before a human looks, and it works well. Triage agent PRs up front, fast-track the trivial ones, and do not let a person sink an hour into a sprawling change the agent will abandon as soon as you push back.

Raise the bar for what you will even review. The fix for being buried is not locking down the repository, it is refusing to review changes that arrive without evidence. Require, before review: a statement of what the change is for, a diff that is not 3,500 lines with no comments, the test output, and proof it was actually run. This is how you stop being the first human to read the code. You push the intent-reconstruction work back onto whoever submitted it, where

it is cheap, rather than absorbing it yourself, where it is expensive.

Keep PRs small, deliberately. Agent PRs run large, 51% larger on average in the Faros data, and reviewer engagement is one of the strongest predictors that a PR merges at all. A large unreviewable PR gets rejected outright or, worse, rubber-stamped. Instruct your agents to produce small commits. A diff a human can actually read is now a design constraint, not a courtesy.

Read the test changes more carefully than the code. This is the agent failure mode to watch. The agent changes behavior, then "fixes" the test by rewriting the assertion to match the new, broken behavior. A green check over 200 edited tests means nothing until you have confirmed the edits were correct. Treat any diff that rewrites many tests as a flag and read those first. Mutation testing earns its place here: coverage tells you a line ran, mutation testing tells you whether the test would notice if that line were wrong.

Treat CI as the wall that does not move. Watch for the patterns GitHub now warns reviewers about: removed tests, skipped lint, lowered coverage thresholds, a duplicated helper that already exists elsewhere, and untrusted input flowing into a prompt. That last one deserves emphasis, because agent-built features are a fresh source of prompt injection: if a change pipes user-controlled text into an LLM call without thinking about what that text can instruct the model to do, the vulnerability is not visible in the diff, it is latent in the data that will arrive later. Agents will also weaken CI to make themselves pass, not maliciously, just gradient descent finding the cheapest path to green. Deterministic gates are the one part of the pipeline that

cannot be talked out of their verdict by a confident paragraph, so keep them strict.

A human owns the merge. A model cannot be paged and cannot be held responsible for what it shipped, so whoever clicks merge owns it. When an AI review says “looks good” in a calm, confident voice, it is handing you confidence it has not necessarily earned. Treat every AI review as a sensor, not a verdict: data, not a decision.

If you are solo with no users, the tiering, the test-change discipline and CI are most of what you need; the rest is overhead until people show up. If you are the large organization, all of it is the baseline, and the triage and intake bar are the difference between a review process that scales and one that quietly collapses.

What this means if you run a team

The bottleneck is no longer how fast you write code, it is how fast a trusted human can be confident in a review. Cutting the people who provide that confidence because “AI made us faster” simply converts the saving into future incidents.

The binding constraint on shipping is no longer how fast you can write code. It is how fast a trusted human can be confident a change is correct. Any plan that treats generation as the bottleneck and review as free will quietly stall, with the velocity dashboard staying green the whole way.

The Faros report is direct about this: QA and review work rises even as output rises, so reducing engineering headcount because “AI made us faster” is dangerous unless

you have closed the review gap first. The senior-engineer tax, review time up by triple digits, falls hardest on the people you can least afford to bottleneck, and it is invisible to any metric that only counts merged PRs.

Open source maintainers hit this wall first and hardest. The steady stream of plausible but hollow contributions costs real triage time even when it is well-intentioned, and that is the canary. Companies are next. The ones handling it well treat review capacity as a real resource to be measured, protected and spent deliberately, not as slack that AI has freed up.

Writing got cheap, understanding didn't

Code review did not become less important when agents arrived. It became the central activity. Writing code is increasingly solved and getting cheaper by the month; the durable advantage is the system that lets you trust what was written.

Do not take the one-size answer in either direction. If you are solo with no users, the enterprise horror stories about churn and duplication are a future risk, not today's fire, so lean on your tests, review what matters, and stay honest that the deferred work is still owed. If you maintain something large for many people, every alarming number here is about you, and the only thing that holds is a tiered, evidence-required, deliberately heterogeneous review process with a human owning the merge.

What is constant across the whole spectrum is the underlying economics. We made writing cheap, and understanding stayed exactly as expensive as it has always

been. The teams that do well over the next few years will not be the ones generating the most code, they will be the ones who built a review system they can actually trust, and who never confuse “the tests passed” with “a person understands what this does and why”.

Or, as Simon Willison keeps putting it, your job is to deliver code you have proven to work. Agents have not changed that. They have made the proving the center of the job rather than an afterthought, and I think that is a good trade. Understanding a system well enough to stand behind it is the most durable and most interesting skill in software, and there has never been a better time to get extraordinarily good at it.